

EXAMEN C++ – VERSION C

Master Sciences pour l'Ingénieur

Durée : 1 heure

Consignes :

- Répondez aux **2 exercices** ci-dessous
 - La durée totale est d'**une heure**
 - Vous pouvez utiliser du papier brouillon pour le pseudo-code
 - **Compilabilité obligatoire** : votre code doit compiler sans erreur
 - Lisibilité et commentaires sont appréciés
-

Exercice 1 : Hiérarchie de Véhicules [10 points]

Contexte

Vous devez modéliser une flotte de véhicules. Chaque véhicule a une consommation de carburant différente selon son type. L'objectif est de calculer le coût d'un trajet et d'afficher les informations de chaque véhicule.

Questions

1. [3 points] Créez une classe abstraite `Vehicle` avec :
 - Des membres protégés : `string brand` (marque) et `double tankCapacity` (capacité du réservoir en litres)
 - Une méthode virtuelle pure `double fuelConsumption()` retournant la consommation aux 100 km (en L/100km)
 - Une méthode virtuelle `void displayInfo()` affichant la marque et la capacité du réservoir
 - Un destructeur virtuel
2. [3 points] Créez deux classes concrètes :
 - `Car(string brand, double tankCapacity, double consumption)` avec :
 - `fuelConsumption()` retourne `consumption` (valeur stockée)
 - `displayInfo()` affiche la marque, la capacité et le type "Voiture"
 - `Truck(string brand, double tankCapacity, double loadTons)` avec :
 - `fuelConsumption()` retourne $20.0 + 2.5 \times \text{loadTons}$ (camion chargé)
 - `displayInfo()` affiche la marque, la capacité, le type "Camion" et le tonnage

3. **[2 points]** Surchargez l'opérateur `>` pour comparer deux véhicules selon leur consommation. L'expression `v1 > v2` doit retourner `true` si `v1` consomme **plus** que `v2`.
4. **[2 points]** Écrivez un `main()` qui :
- Crée une `Car` de marque "Toyota", réservoir 50L, consommation 6.5 L/100km
 - Crée un `Truck` de marque "Renault", réservoir 300L, charge 5.0 tonnes
 - Affiche les infos de chaque véhicule et leur consommation
 - Affiche lequel des deux consomme le plus en utilisant l'opérateur `>`

Exercice 2 : File Générique (Queue) [10 points]

Contexte

Vous devez créer un template de classe générique `Queue<T>` (file/queue FIFO) pour gérer des éléments de n'importe quel type. Contrairement à une pile, les éléments sont insérés en queue et retirés en tête.

Questions

1. [4 points] Créez un template de classe `Queue<T>` avec :
 - Un membre privé `vector<T> data` pour stocker les éléments
 - Une méthode `void enqueue(const T& element)` pour ajouter un élément en queue
 - Une méthode `T dequeue()` qui retourne et supprime l'élément en tête (indice 0)
 - Une méthode `T front()` qui retourne l'élément en tête sans le supprimer
 - Une méthode `bool isEmpty()` qui retourne `true` si la file est vide
 - Une méthode `int size()` qui retourne le nombre d'éléments

2. [2 points] Surchargez l'opérateur `!=` pour comparer deux files :
 - Deux files sont **différentes** si elles n'ont pas les mêmes éléments dans le même ordre
 - Vous ne pouvez pas modifier les files pendant la comparaison

3. [4 points] Écrivez un `main()` qui :
 - Crée une `Queue<int>` et y ajoute les entiers 1, 2, 3 avec `enqueue`
 - Crée une `Queue<string>` et y ajoute "Bonjour", "le", "monde"
 - Affiche la taille et l'élément en tête (`front`) de chaque file
 - Retire et affiche les éléments de la `Queue<int>` dans l'ordre FIFO
 - Crée une deuxième `Queue<string>` identique à la première et vérifie avec `!=` qu'elles sont bien égales (résultat attendu : `false`)

Notes

- Incluez les headers nécessaires : `<iostream>`, `<string>`, `<vector>`
- `using namespace std;` est autorisé
- Utilisez `const T&` dans les paramètres quand approprié
- Pour `dequeue()` et `front()`, vous pouvez lever une exception ou utiliser `exit(1)` si la file est vide
- Pour un template, la définition doit être dans le même fichier que la déclaration
- N'oubliez pas le destructeur virtuel dans la classe abstraite